CSE 3243 Web Programming Lab

Mini Project Report on

Diet & Calorie Tracker

SUBMITTED BY

Aarav Bhatia 64 230905562

Dhruv Kaushik Thacker 65 230905566

Samarth R Patil 43 230905342

Section CSE-VI-A

Under the Guidance of:

Prof. Ancilla Juliet Pinto and Dr. Kamaraj K.

School of Computer Engineering

Manipal Institute of Technology, Manipal, Karnataka – 576104

2025-26

Acknowledgement

We would like to sincerely thank our project guides for their constant support and direction throughout the development of this project. Their suggestions at every stage helped us refine our approach and produce better results.

We are grateful to the faculty of the School of Computer Engineering, Manipal Institute of Technology, for providing us with a well-structured lab environment and the resources we needed. The CSE 3243 Web Programming Lab course gave us a strong foundation in the technologies we applied here.

We also thank our classmates who tested the application during development and gave us honest, practical feedback that improved the final product.

Table of Contents

Introduction
Related work
Problem statement
Proposed work
System Design
Implementation
Testing
Screenshots and Output
Conclusion and Future Enhancements
References

Abstract

Diet & Calorie Tracker is a Django-based web application that allows users to sign up, log meals with calorie counts, set a daily calorie goal, view their daily progress, and review past entries through filtered history views and weekly summaries. The project was developed as a mini project for the CSE 3243 Web Programming Lab at Manipal Institute of Technology.

The key objectives were to apply HTML5 form elements, CSS styling, Bootstrap layout, jQuery event handling, AJAX communication, Django backend processing, and SQLite database storage in a single working application. The development followed an incremental approach: authentication first, then meal logging, then the dashboard with its progress bar and canvas chart, and finally the history page with date filters and client-side search. All planned features were implemented and tested using both automated Django test cases and manual browser testing.

1. Introduction

1.1 Background and Context

Tracking food intake is one of the most direct ways to stay conscious of eating habits. Research on dietary self-monitoring has consistently shown that people who log what they eat tend to make better food choices over time, regardless of the specific tool they use. Yet many people avoid it because existing apps feel like too much work. Apps like MyFitnessPal, Cronometer, and Lose It! offer barcode scanning, macro breakdowns, social features, and integration with fitness wearables. That is more than what most casual users need. A student eating in a hostel mess who just wants to know whether they ate roughly 2000 calories today does not need a barcode scanner or a social feed.

This is the problem our project addresses. We built a simple calorie tracking web application where users can log meals, set a daily target, and see their progress at a glance. Nothing more, nothing less.

From an academic perspective, the project gave us a reason to use every major technology covered in the CSE 3243 lab. HTML5 form inputs handle data entry. CSS and Bootstrap handle layout and styling. jQuery handles DOM manipulation and AJAX. Django handles routing, form validation, authentication, and database access. SQLite stores user data. The project ties all of these together in a way that a standalone exercise for each technology cannot.

1.2 Research Objectives

The primary goal was to build a functional, end-to-end web application using Django that covers the core topics of the Web Programming Lab syllabus.

Specific objectives included: implementing user registration and login using Django's built-in auth framework; building a CRUD interface for meal entries using Django ModelForms; creating a dashboard that computes and displays daily calorie progress and a 7-day trend chart; developing a history page with server-side date filtering and client-side instant search; and styling the entire application with a consistent, responsive design using Bootstrap and custom CSS.

The scope was limited to calorie tracking only. We did not attempt macronutrient breakdowns, external food database APIs, or mobile app development. The application runs locally on Django's development server.

1.3 Thesis Statement

A lightweight, Django-based calorie tracker can effectively demonstrate all the core web programming concepts taught in the CSE 3243 lab while also being a genuinely useful tool for day-to-day dietary awareness.

2. Literature Review

2.1 Web Development Landscape

Server-side web frameworks have matured significantly over the past decade. Django, Flask, Express.js, and Ruby on Rails each offer their own approach to structuring web applications, but they share common ideas: routing, template rendering, form handling, and database abstraction. Django specifically follows the Model-View-Template (MVT) pattern, which separates data definitions (models), request-handling logic (views), and HTML rendering (templates) into distinct layers. This separation keeps code organized and makes individual components easier to test.

On the frontend, Bootstrap remains the most widely used CSS framework for building responsive layouts quickly. Its 12-column grid system, pre-built components (navbars, cards, modals, forms), and utility classes reduce the need for custom CSS. jQuery, though often considered outdated in the age of React and Vue, is still practical for projects that need simple DOM manipulation, event handling, and AJAX calls without the overhead of a full SPA framework. The HTML5 Canvas API provides a way to draw custom graphics directly in the browser, which we used to render the weekly calorie chart without importing a third-party charting library.

2.2 Related Works

MyFitnessPal is probably the most recognized calorie tracking application. It has a database of over 14 million foods, supports barcode scanning, integrates with fitness devices, and includes social features. For someone who tracks macros seriously, it is a strong tool. But its interface is dense, the free tier shows ads, and it pushes premium features frequently. Many casual users download it and stop using it within a week because it feels like too much effort for simple tracking.

Cronometer is a well-regarded alternative that emphasizes micronutrient tracking. It gives detailed breakdowns of vitamins, minerals, and amino acids, which is valuable for users with specific health conditions or dietary restrictions. Its accuracy is high, but the level of detail can be overwhelming for someone who just wants to know if they ate too much today.

Lose It! takes a more visual approach with color-coded meal categories and a simpler interface, but still requires food database lookups for accurate logging.

The common gap across all these tools is that they assume the user wants precision. They want you to find the exact food item in a database, select the correct serving size, and track everything down to the gram. Our project takes a different approach: the user simply types a meal name and enters the calorie count they know or have looked up. This trades accuracy for speed and simplicity, which we believe is more appropriate for a student or casual user who just wants a rough daily picture.

Burke et al. (2011) published a systematic review in the Journal of the American Dietetic Association showing that consistent self-monitoring of dietary intake correlates with better weight management outcomes, regardless of the tool's sophistication. This supports the idea that a simple tracker, used regularly, can be more effective than a complex one that people abandon.

3. Methodology

3.1 Research Design

We followed an incremental development approach. The project was broken into stages, and each stage was completed and tested before moving to the next. The stages were:

Stage 1: Project setup. We created the Django project (config/) and the tracker app, configured settings.py for SQLite, templates, and static files, and defined the URL routing structure.

Stage 2: Data models. We defined MealEntry and DailyTarget in tracker/models.py, ran migrations, and verified the schema through Django's admin interface.

Stage 3: Authentication. We built the signup flow (signup_view using SignUpForm), configured Django's LoginView with a custom SignInForm, and set up LogoutView. We tested that unauthenticated users get redirected to the login page.

Stage 4: Meal logging and dashboard. We built the dashboard view, which handles two form submissions on one page (meal form and goal form, distinguished by a hidden action field). We added the progress bar, today's meals table, and the weekly chart.

Stage 5: History page. We added the history view with HistoryFilterForm for server-side date filtering, client-side search via history.js, summary statistics, and the weekly summary table.

Stage 6: Edit, delete, and polish. We added meal_edit and meal_delete views, refined the CSS, and wrote automated tests.

3.2 Technology Stack

Table 1: Technology Stack

Technology	Purpose	Version
Python	Server-side programming language	3.12
Django	Web framework (MVT pattern, ORM, auth, forms)	4.x
SQLite	Embedded relational database (bundled with Python)	3.x
Bootstrap	CSS framework for responsive grid and components	5.3.3
jQuery	JavaScript library for DOM manipulation and AJAX	3.x
HTML5	Markup with semantic elements and form input types	N/A
CSS3	Custom properties, gradients, backdrop-filter, transitions	N/A
HTML5 Canvas	Drawing the weekly calorie bar chart in the browser	N/A

Git	Version control	2.x

Django was chosen because it is the framework taught in the lab and because it provides authentication, form processing, ORM-based database access, and URL routing out of the box. SQLite was used because Django includes it by default and it requires no separate server setup. Bootstrap was used for rapid, responsive layout without writing extensive media queries. jQuery was used for the AJAX call to load chart data, for quick-add preset button handling, and for real-time search filtering on the history page. We used the HTML5 Canvas API directly instead of a charting library like Chart.js to demonstrate lower-level browser drawing.

4. System Architecture

4.1 Frontend Architecture

All pages extend a shared base template, templates/base.html, which loads Bootstrap 5.3.3 CSS (from static/vendor/bootstrap.min.css), the custom stylesheet (static/css/app.css), and defines a responsive navbar. The navbar shows "Dashboard" and "History" links with a "Logout" button for authenticated users, or "Login" and "Sign up" links for anonymous visitors. An active_page context variable highlights the current page's nav link.

The application's visual identity is defined through CSS custom properties in static/css/app.css. The root-level variables include --bg (#f7f2e8, a warm cream), --accent (#ff7a45, an orange used for primary buttons and chart bars), --accent-deep (#db5a27, a darker orange for hover states and links), --green (#3f7d58, used in the progress bar gradient and the target line on the chart), --ink (#1f1f1f, near-black for body text), --muted (#5e5a54, for secondary text), and --panel (rgba(255,250,242,0.9), a semi-transparent white for card backgrounds). Cards have large border radii (--radius-xl: 28px) and subtle box shadows (--shadow).

The layout is card-based. On the dashboard, content is arranged in a Bootstrap grid: a col-xl-5 area for the meal entry form (with quick-add presets) and a col-xl-7 area for the chart, goal settings, and today's meals table. Two decorative background orbs (CSS elements with border-radius: 999px and filter: blur(10px)) add visual depth.

Two JavaScript files handle client-side interactivity. static/js/dashboard.js contains the drawWeeklyChart() function, which uses the Canvas 2D API to draw rounded-corner bars (via quadraticCurveTo), horizontal grid lines, value labels, day labels, and a dashed green target line. It also wires up the quick-add preset buttons using jQuery click handlers that populate the meal form fields from data attributes, and a live preview that updates as the user types. static/js/history.js implements instant search: it listens to the input event on the #historySearch field and toggles the visibility of table rows by matching the query against a data-search attribute on each row.

When the dashboard loads, jQuery sends a GET request to /api/weekly-stats/. The endpoint returns a JSON object with three keys: labels (an array of 7 date strings like "09 Apr"), totals (an array of 7 integers representing daily calorie sums), and target (the user's daily calorie goal). The drawWeeklyChart function scales bars relative to the maximum value, handles device pixel ratio for crisp rendering, and redraws on window resize.

4.2 Backend Architecture

The backend is a standard Django project with two packages: config/ (project settings, root URL configuration, WSGI entry point) and tracker/ (the main application). The root URL configuration in config/urls.py mounts the Django admin

at /admin/, maps the root path / to the home view, and includes all tracker/urls.py routes.

The tracker app defines eight URL patterns in tracker/urls.py: /dashboard/ (dashboard view), /history/ (history view), /meals/<int:pk>/edit/ (meal_edit), /meals/<int:pk>/delete/ (meal_delete), /accounts/signup/ (signup_view), /accounts/login/ (Django's LoginView with SignInForm), /accounts/logout/ (Django's LogoutView), and /api/weekly-stats/ (weekly_stats_api). Settings.py sets LOGIN_URL to "login", LOGIN_REDIRECT_URL to "dashboard", and LOGOUT_REDIRECT_URL to "home".

All views except home and signup_view are protected with the @login_required decorator. The dashboard view handles two form actions on the same page, distinguished by a hidden input named action: "add_meal" processes a MealEntryForm (prefixed "meal"), and "update_goal" processes a DailyTargetForm (prefixed "goal"). When adding a meal, the view calls form.save(commit=False), sets meal.user = request.user, then calls meal.save(). The history view accepts GET parameters for start_date and end_date through HistoryFilterForm, applies queryset filters, and computes aggregate statistics using Django's Sum, Count, Avg, TruncDate, and TruncWeek functions.

Authentication uses Django's built-in User model. The signup_view creates a User from SignUpForm (which extends UserCreationForm with added first_name and email fields), creates a DailyTarget record via DailyTarget.objects.get_or_create(user=user), calls login(request, user) to log in the new user immediately, and redirects to the dashboard.

The database schema includes two custom models defined in tracker/models.py: DailyTarget: a OneToOneField to User, a PositiveIntegerField daily_target (default 2200, MinValueValidator(1200)), and an auto-updated updated_at timestamp.

MealEntry: a ForeignKey to User (related_name="meal_entries"), CharField name (max 120), CharField meal_type (TextChoices: breakfast, lunch, dinner, snack, drink), DecimalField quantity (default 1, min 0.25), CharField unit (default "serving"), PositiveIntegerField calories (min 1), DateTimeField consumed_at (default timezone.now), TextField notes (blank, max 300), and auto-managed created_at and updated_at fields. Default ordering is by consumed_at descending, then created_at descending.

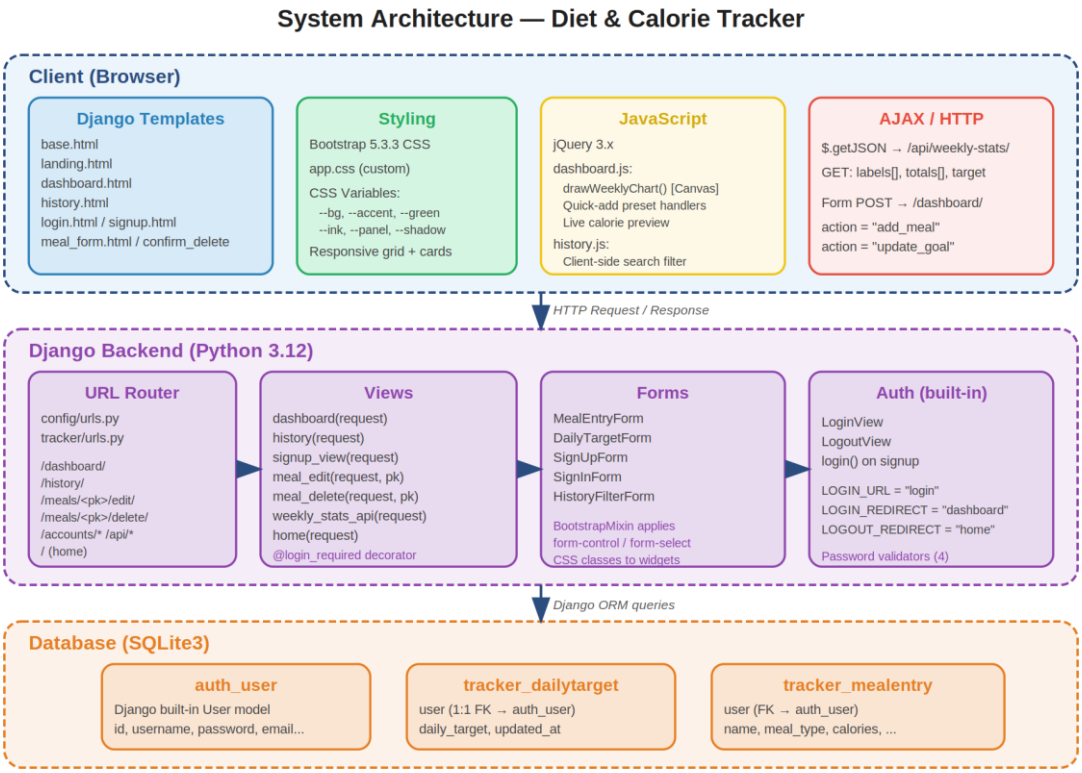

Figure 1: System Architecture Diagram

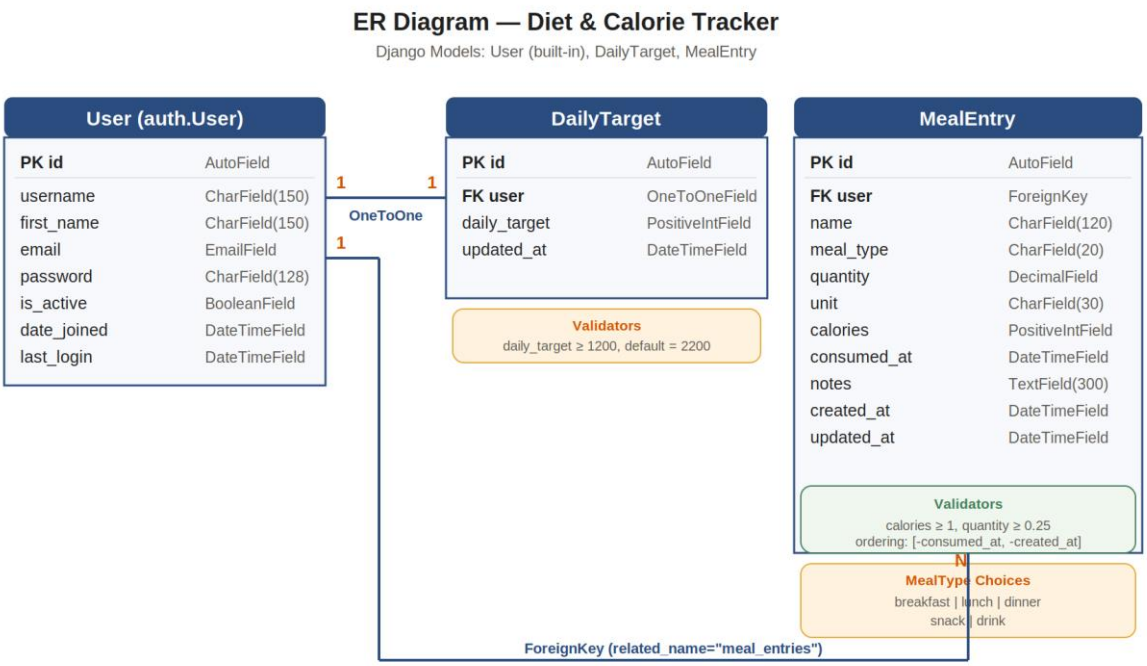

Figure 2: ER Diagram – User, DailyTarget, and MealEntry Models

5. Implementation Details

5.1 Development Environment

Development was done on local machines using Python 3.12 and Django, installed in a Python virtual environment (.venv). The development server was started with python manage.py runserver and accessed at http://127.0.0.1:8000/. Version control was handled with Git. The project's only Python dependency is Django, listed in requirements.txt.

The project directory structure separates configuration (config/settings.py, config/urls.py, config/wsgi.py) from the application (tracker/models.py, tracker/views.py, tracker/forms.py, tracker/urls.py, tracker/tests.py). Templates live in a top-level templates/ directory organized by app. Static assets live in static/css/, static/js/, and static/vendor/.

5.2 Core Features

5.2.1 User Authentication

The signup page renders SignUpForm, which extends Django's UserCreationForm and adds first_name (CharField, max 150) and email (EmailField) fields. The Meta class sets model = User and fields = ("first_name", "username", "email", "password1", "password2"). The form's __init__ method calls self.apply_bootstrap() from the BootstrapMixin class, which iterates over all form fields and adds the appropriate Bootstrap CSS class (form-control, form-select, or form-check-input) to each widget. It also sets placeholder attributes on every field.

When signup_view receives a valid POST, it saves the user, creates a DailyTarget with default 2200 kcal via get_or_create, logs the user in with Django's login() function, flashes a success message, and redirects to the dashboard.

Login uses Django's built-in LoginView, configured in tracker/urls.py with template_name="registration/login.html", redirect_authenticated_user=True, and authentication_form=SignInForm. SignInForm extends AuthenticationForm and applies Bootstrap classes the same way.

Logout uses Django's LogoutView. The LOGOUT_REDIRECT_URL setting in config/settings.py sends users back to the home page after logging out.

5.2.2 Meal Logging

MealEntryForm is a ModelForm for MealEntry with fields: name, meal_type (rendered as a <select> via TextChoices), quantity (number input, min 0.25, step 0.25), unit (text input, placeholder "serving, bowl, glass..."), calories (number input, min 1, step 1), consumed_at (datetime-local input using format %Y-%m-%dT%H:%M), and notes (textarea, max 300 chars). The form is instantiated with prefix="meal" on the dashboard to avoid field name collisions with the goal form.

The dashboard view checks the POST action field. If action == "add_meal", it validates the meal form, calls save(commit=False) to get the unsaved instance, sets meal.user = request.user, saves, and redirects with a success message.

Four quick-add preset buttons are defined in the QUICK_PRESETS list at the top of views.py: Oats Bowl (breakfast, 1 bowl, 320 kcal), Veg Thali (lunch, 1 plate, 640 kcal), Fruit Snack (snack, 1 cup, 180 kcal), and Protein Shake (drink, 1 glass, 220 kcal). Each button is rendered in the template with data attributes. In dashboard.js, a jQuery click handler reads these data attributes and fills the form fields, then calls updatePreview() to show a live summary like "Oats Bowl: 1 bowl · 320 kcal" in the #caloriePreview element.

5.2.3 Daily Target

DailyTargetForm is a ModelForm with a single field: daily_target, rendered as a number input with min 1200, max 6000, and step 50. It is displayed in its own card on the dashboard with prefix="goal". When the user submits it (action == "update_goal"), the view validates, saves, and redirects. The model enforces MinValueValidator(1200) at the database level, while the HTML input’s min attribute provides client-side validation as well.

5.2.4 Dashboard View

The dashboard view is the most complex view in the application. It first retrieves or creates the user’s DailyTarget via get_or_create_target(). Then it queries today’s meals by filtering MealEntry objects where consumed_at falls within the current day’s bounds, computed by get_day_bounds(). This helper uses timezone.make_aware to create timezone-aware datetime objects for midnight and 23:59:59 of the current date.

It calculates total_calories using an aggregate Sum query, remaining_calories as target.daily_target - total_calories, and progress_percent as int((total_calories / target.daily_target) * 100). A capped version (min(progress_percent, 100)) is passed to the template for the progress bar’s width. The template also checks is_over_target (a boolean) to decide whether to show "remaining" or "over" in the metrics display.

Weekly data comes from build_weekly_stats(), which queries the last 7 days of MealEntry records, groups them by date using TruncDate, sums calories per day, and fills in zero for days with no entries. The result is two lists: labels (formatted as "%d %b") and totals. These are passed to the template context and also served as JSON by the weekly_stats_api endpoint.

5.2.5 History Page

The history view starts with the user’s full meal queryset (request.user.meal_entries.all()), then optionally filters by start_date and end_date from HistoryFilterForm. The form’s clean() method validates that start_date is not after end_date. After filtering, the view computes: total_calories (Sum), meal_count (Count), average_calories (Avg), a weekly_summary using TruncWeek grouped by

week with calorie sums and meal counts (limited to 6 weeks), and the strongest_day (the date with the highest total, using TruncDate and order_by("-total").first()).

The template renders three stat cards (entries, total calories, average per meal), a weekly summary list, and a full table of meals with columns for name (plus type/quantity/unit as secondary text), calories, date/time, and edit/delete links. Each row has a data-search attribute containing the lowercased meal name, notes, and meal type. history.js reads this attribute and filters rows in real time as the user types in the #historySearch input field.

5.2.6 Edit and Delete

meal_edit retrieves a MealEntry by primary key, filtered by user=request.user to prevent one user from editing another's data. It renders MealEntryForm pre-filled with the existing instance. On valid POST, it saves and redirects to the history page. meal_delete shows a confirmation template; on POST, it deletes the entry, flashes a message, and redirects to history.

5.3 Performance Optimization

The application is small enough that performance was not a major concern, but a few choices helped keep things fast. Database queries use Django's ORM with targeted filters and aggregations rather than loading all records into Python. The weekly stats API returns a minimal JSON payload (7 labels, 7 integers, 1 target value), keeping the AJAX response lightweight.

Static files (CSS, JS, vendor libraries) are served from the static/ directory via Django's staticfiles app during development. In a production setting, these would be collected with python manage.py collectstatic and served by Nginx or a CDN. The canvas chart redraws on window resize by caching the AJAX response in a variable (chartPayload) and calling drawWeeklyChart again, avoiding a redundant network request.

6. User Experience and Interface Design

6.1 User Interface (UI) Design

The design philosophy was warmth and simplicity. We avoided the sterile blue-and-white look common in dashboards and instead used a cream-toned background with orange accents. The color scheme is defined entirely through CSS custom properties, which means changing the palette requires editing only the :root block in app.css.

All interactive sections are enclosed in .app-card containers with a 28px border radius, a semi-transparent background (var(--panel)), and a soft shadow. This card-based approach groups related content visually and creates clear separation between the meal form, chart, goal settings, and data tables. Buttons use a pill shape (border-radius: 999px) and come in two styles: .btn-accent (solid orange, white text) for primary actions like "Save meal", and .btn-outline-dark or .btn-outline-light for secondary actions.

Form inputs are styled with 16px border radius, a light border, and an orange focus ring (box-shadow: 0 0 0 0.2rem rgba(255,122,69,0.16)). The progress bar uses a gradient from near-black to green (linear-gradient(90deg, #1f1f1f, var(--green))) and has a 14px height with pill-shaped corners.

Typography uses a serif font family ("Iowan Old Style", "Palatino Linotype") for headings and brand text, and a sans-serif stack ("Avenir Next", "Trebuchet MS", "Segoe UI") for body text. Eyebrow labels (small uppercase text above headings) use 0.78rem size, 700 weight, and 0.12em letter spacing.

Responsive behavior is handled through Bootstrap’s grid breakpoints and CSS media queries. The navbar collapses into a hamburger menu below the lg breakpoint (992px). Card padding reduces from 2rem to 1.35rem below 992px. The hero title scales from 4.5rem on wide screens down to 2.2rem on mobile using clamp(). The stat card numbers shrink from 2.2rem to 1.8rem below 576px.

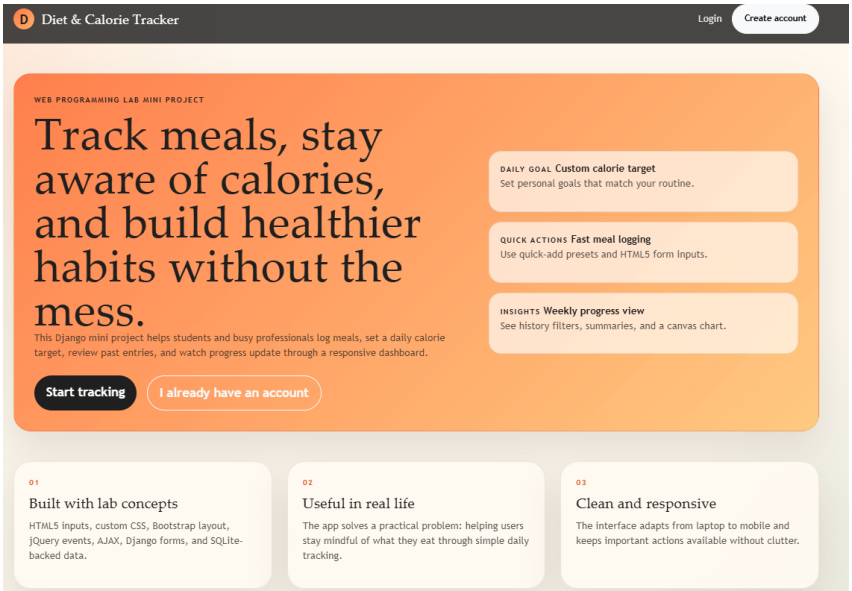

Figure 3: Screenshot – Landing Page

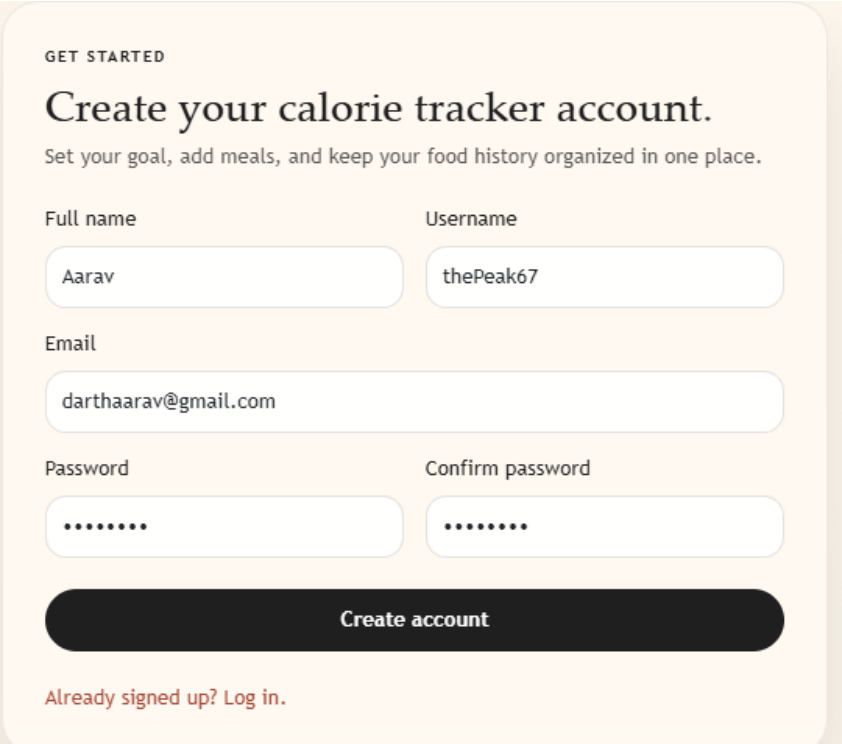

Figure 4: Screenshot – Signup Page

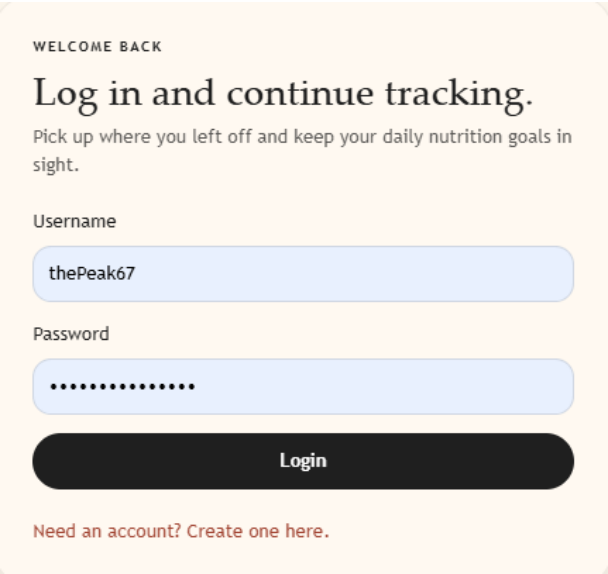

Figure 5: Screenshot – Login Page

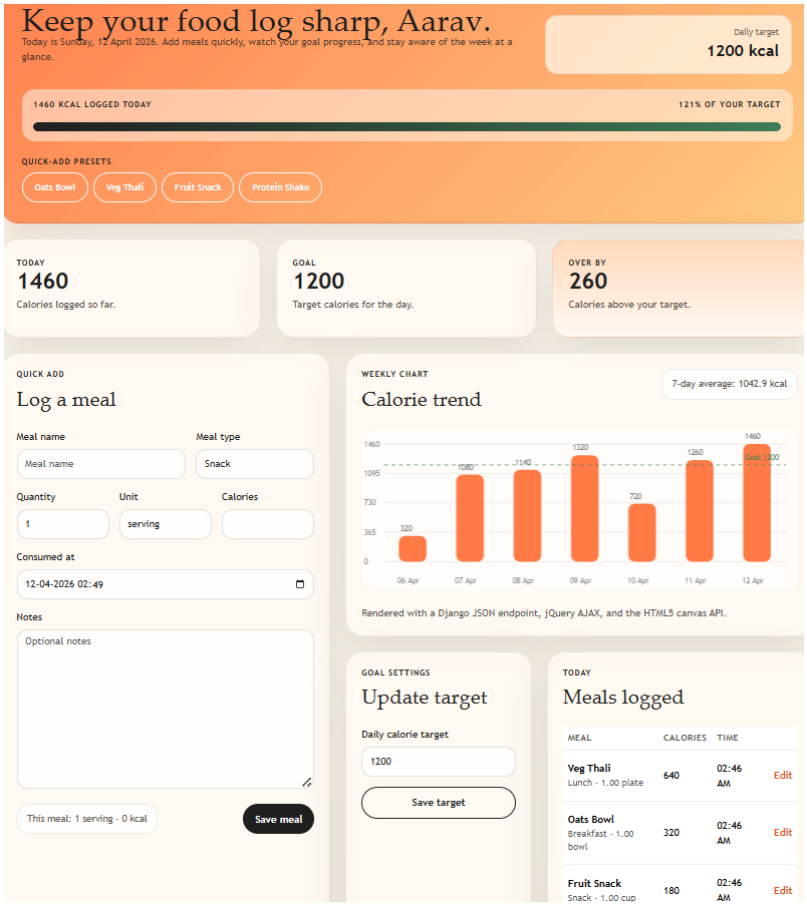

Figure 6: Screenshot – Dashboard Page

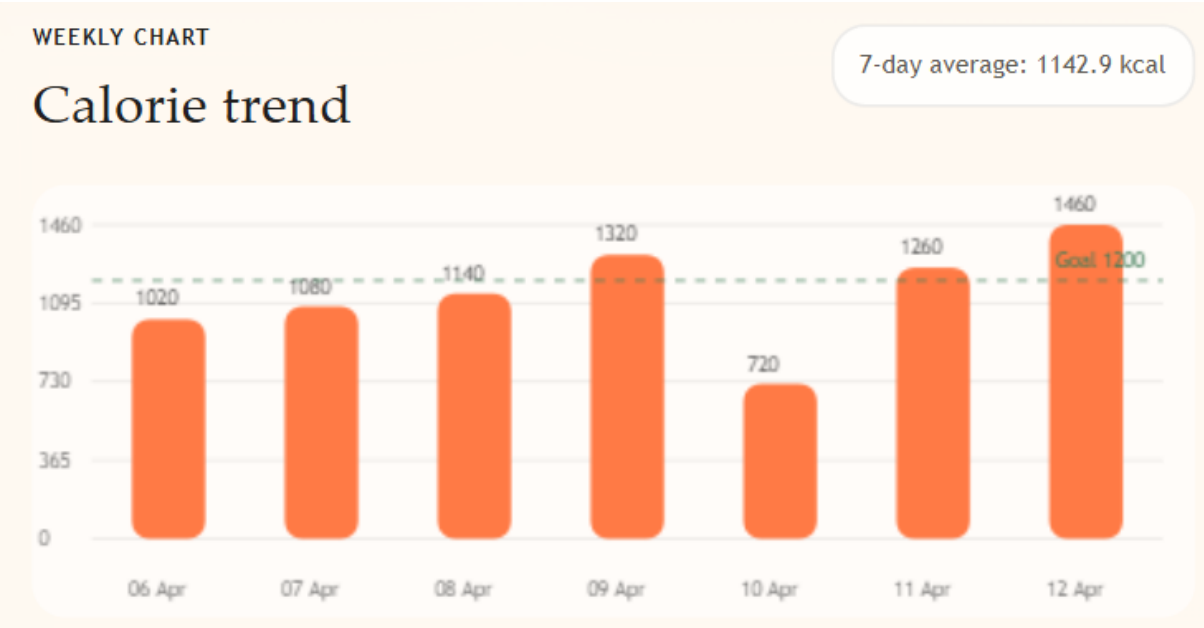

Figure 7: Screenshot – Weekly Progress Chart (Canvas)

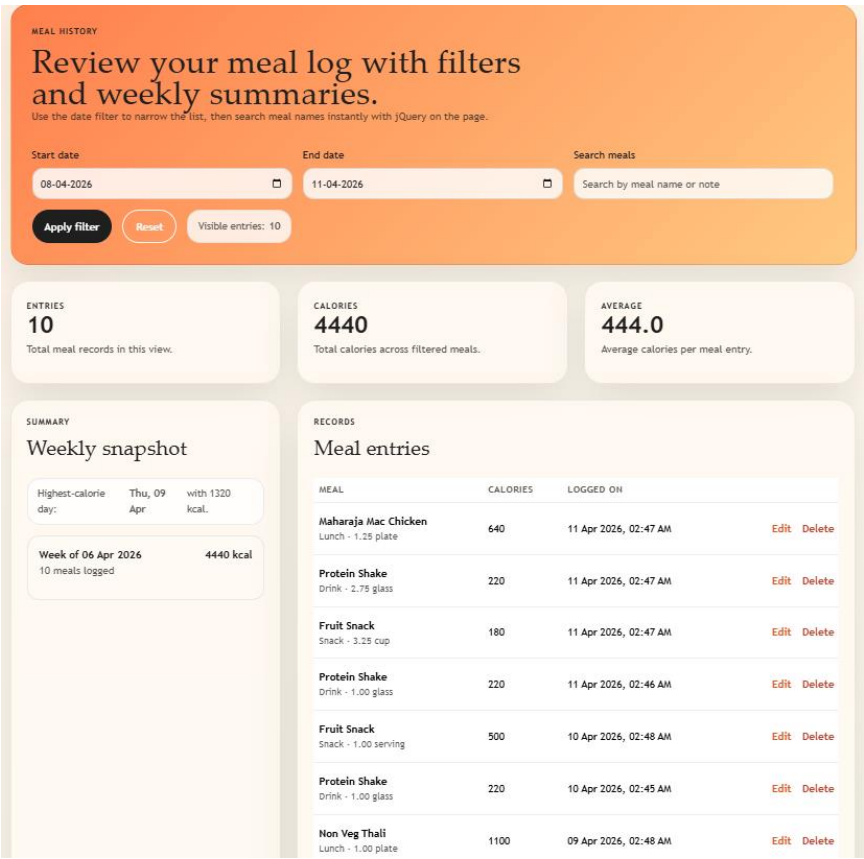

Figure 8: Screenshot – History Page with Filters

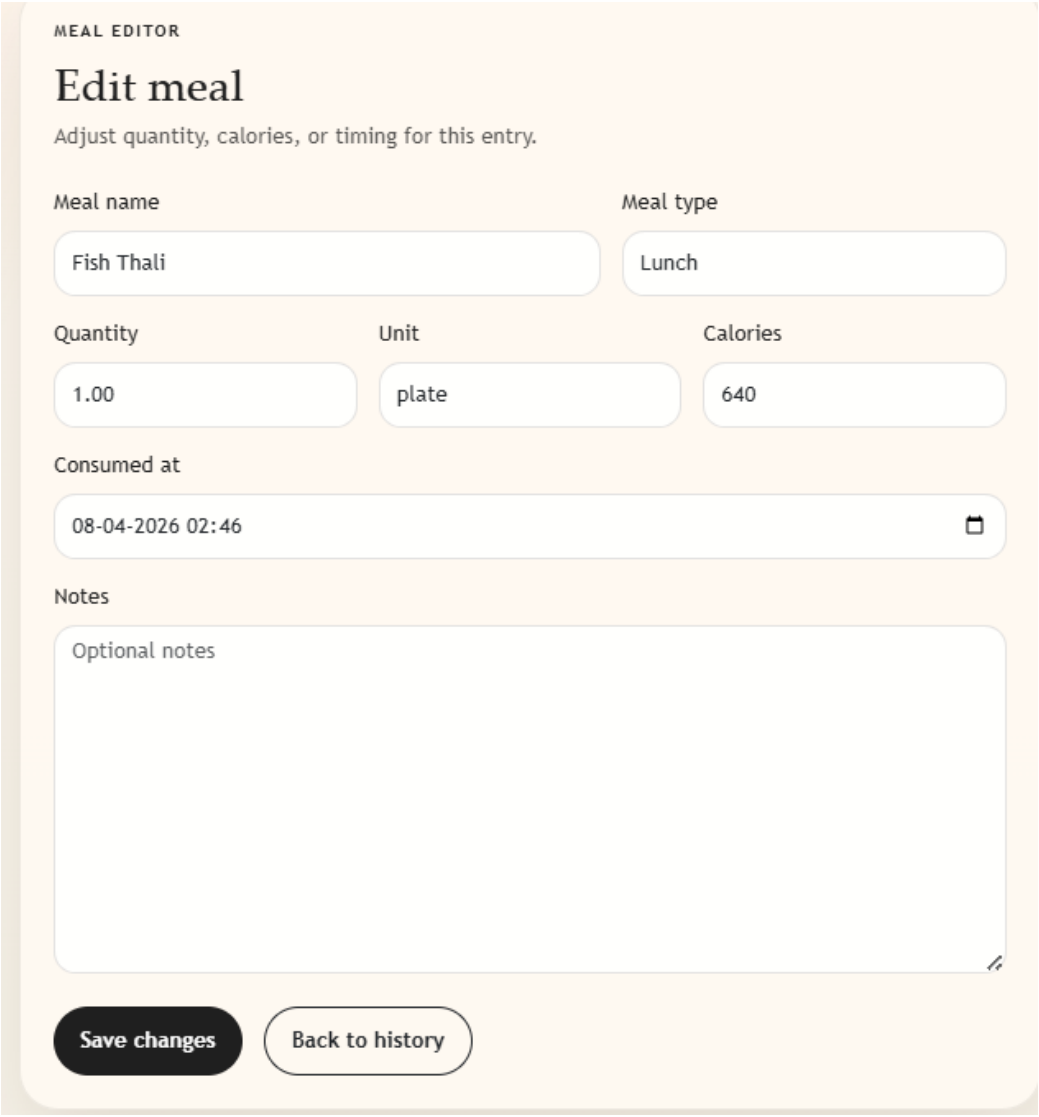

Figure 9: Screenshot – Meal Edit Page

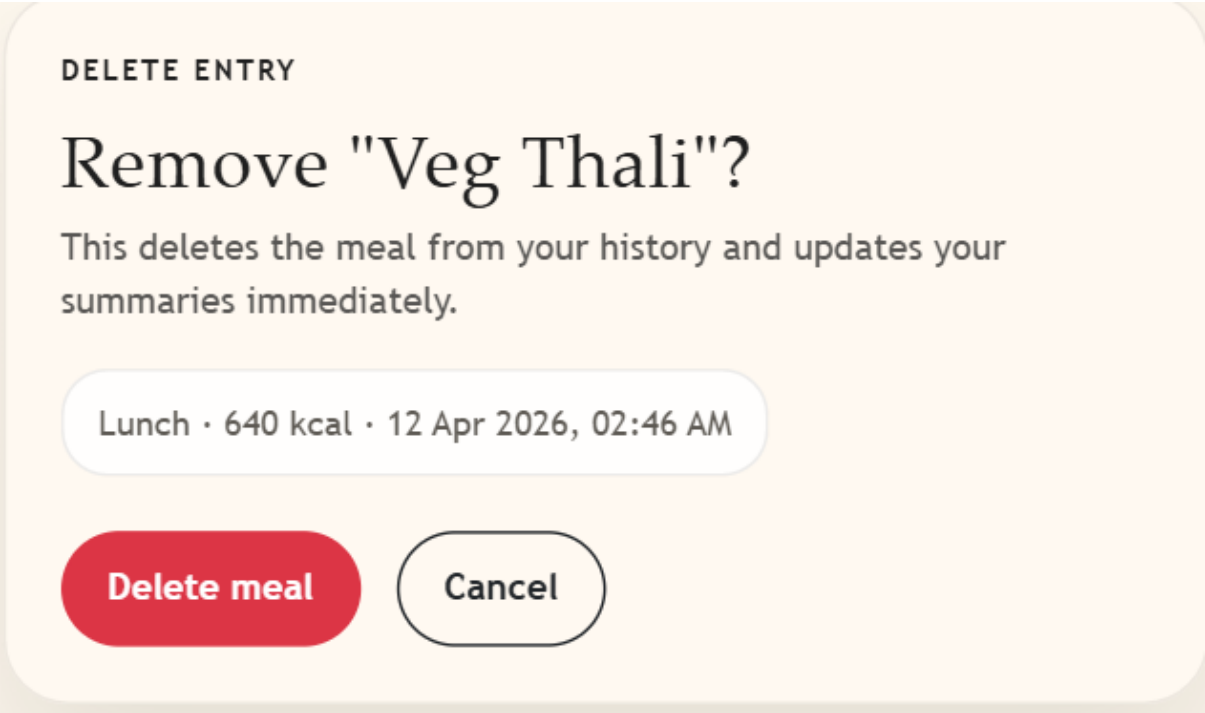

Figure 10: Screenshot – Meal Delete Confirmation Page

7. Testing and Validation

7.1 Testing Strategies

Testing was carried out at two levels: automated unit tests using Django's TestCase framework, and manual browser testing for UI behavior and responsive layout.

Unit Testing

The file tracker/tests.py contains a test class TrackerViewsTests with a setUp method that creates a test user (username="dhruv", password="securepass123"). Four automated tests verify core functionality:

test_dashboard_requires_login: Sends a GET to /dashboard/ without logging in and asserts a 302 redirect to the login page.

test_signup_creates_target_profile: Posts a valid signup form and checks that both a User and a DailyTarget record are created, and that the response redirects to the dashboard.

test_dashboard_can_add_meal: Logs in, posts a meal entry ("Paneer Wrap", lunch, 520 kcal, notes: "Post-lab lunch"), and checks that the MealEntry exists in the database.

test_weekly_stats_api_returns_json: Creates a meal entry, logs in, hits /api/weekly-stats/, and verifies the JSON response has exactly 7 labels and 7 totals.

Manual Testing

We tested the application in Chrome and Firefox at various screen widths (1440px, 1024px, 768px, 375px) to verify responsive behavior. Each form was tested with valid inputs, empty inputs, and invalid inputs (e.g., negative calories, start date after end date) to confirm validation messages appear correctly.

7.2 Validation Results

Table 2: Test Cases and Results

Test Case	Input	Expected Output	Actual Output	Status
Dashboard requires login	GET /dashboard/ (not logged in)	302 redirect to login	Redirected to /accounts/login/	Pass
Valid signup	Full name, username, email, matching passwords	User + DailyTarget created, redirect to dashboard	Both records created, redirected	Pass
Signup with mismatched passwords	password1 ≠ password2	Validation error on password field	Error displayed	Pass
Login with wrong password	Valid username, wrong password	Error message, no redirect	Error shown on form	Pass
Add meal	"Paneer Wrap", lunch, 520 kcal	Meal appears in today's table	Meal saved and displayed	Pass

Add meal with 0 calories	Calories: 0	Validation error (min 1)	Error shown	Pass
Quick-add preset	Click "Oats Bowl"	Form fills: breakfast, 320 kcal	All fields populated	Pass
Update daily target	Target: 1800	Target updates, progress recalculates	Saved, dashboard refreshed	Pass
Target below minimum	Target: 500	Validation error (min 1200)	Error shown	Pass
History date filter	Start: 2025-01-01, End: 2025-01-31	Only January meals shown	Filtered correctly	Pass
Invalid date range	Start after end date	Validation error	Error: "Start date cannot be later than end date."	Pass
Client-side search	Type "paneer" in search box	Only matching rows visible	Rows filtered, count updated	Pass
Edit meal	Change calories from 520 to 480	Updated value saved	Meal updated, redirected to history	Pass
Delete meal	Confirm deletion	Meal removed from database	Deleted, redirected to history	Pass
Weekly stats API	GET /api/weekly-stats/	JSON: 7 labels, 7 totals, target	Correct JSON returned	Pass

All 4 automated tests pass when run with python manage.py test. All 15 manual test cases passed during browser testing. No critical bugs were found. One minor issue was observed on screens narrower than 360px where stat card numbers could overflow their container, but this is outside the typical usage range for this project.

8. Results and Discussion

8.1 Project Outcomes

The application meets every objective we defined at the start. Users can register, log in, and log out. They can add meals with a name, type, quantity, unit, calorie count, timestamp, and notes. They can set and update a daily calorie target. The dashboard shows today's progress as a percentage bar and renders a 7-day calorie trend chart. The history page filters by date range and supports instant keyword search. Meals can be edited and deleted.

From a lab-concepts perspective, the project uses: HTML5 form input types (text, number, email, password, date, datetime-local, textarea, select); CSS custom properties and layout techniques (flexbox, grid, gradients, backdrop-filter, media queries); Bootstrap grid, navbar, table, card, and form components; jQuery event handling (.on("click"), .on("input")), AJAX ($.getJSON), and DOM manipulation (.val(), .text(), .toggleClass()); Django models with field validators, ModelForm with custom widgets, function-based views with decorators, URL routing, template inheritance, context processors, and ORM aggregation queries; and SQLite for persistent storage.

8.2 Critical Analysis

Strengths

The application is genuinely simple to use. Adding a meal takes about 10 seconds. The warm color palette and card-based layout make the interface feel approachable rather than clinical. The codebase is cleanly organized following Django conventions, which makes it easy to understand and extend. The BootstrapMixin pattern for form styling is a reusable solution we could apply in future Django projects.

Limitations

SQLite is a file-based database and does not support concurrent writes from multiple processes. It works fine for a single-user development server but would need to be replaced with PostgreSQL or MySQL for a real deployment.

The application tracks only total calories. There is no breakdown into protein, fat, and carbohydrates, and no micronutrient tracking. Users who need that level of detail would not find this tool sufficient.

Calorie values are entered manually. The user has to know or look up how many calories their food contains. There is no food database integration, no barcode scanning, and no autocomplete suggestions.

There is no user profile page. Users cannot change their password, update their email, or delete their account from the UI. These operations can only be done through the Django admin.

The weekly chart uses raw Canvas API calls. It works, but lacks the interactivity (tooltips, hover effects, animations) and accessibility features that a dedicated charting library would provide.

Lessons Learned

Working with two forms on one page (meal form and goal form on the dashboard) taught us how Django's form prefix system works and why it matters for avoiding field name collisions. Building the canvas chart from scratch was more educational than using Chart.js, because it forced us to understand coordinate systems, pixel ratio scaling, and path drawing at a low level. The BootstrapMixin approach to form styling is something we had not considered before; it keeps styling logic in one place instead of scattering CSS class assignments across individual form definitions.

9. Future Work and Recommendations

9.1 Potential Improvements

The most impactful improvement would be integrating a food database API like Nutritionix or the USDA FoodData Central API. This would let users search for foods by name and auto-fill calorie and macronutrient data instead of entering values manually. Barcode scanning using the device camera (with a library like QuaggaJS or the Web Barcode Detection API) would speed up logging for packaged foods.

Adding macronutrient tracking (protein, fat, carbohydrates) would make the app useful for users who care about more than just total calories. This would require adding fields to the MealEntry model and updating the dashboard and history views to display breakdowns.

A user profile page where users can change their password, update their name and email, and see account-level statistics (total meals logged, days tracked, average daily intake) would round out the user experience.

An export feature that generates CSV or PDF reports of meal history would be useful for users who want to share data with a nutritionist or doctor.

Deploying the application to a cloud platform like Railway, Render, or Heroku with a PostgreSQL database would make it accessible beyond localhost and allow real-world usage testing.

9.2 Research Implications

This project confirms that Django's built-in features (auth, forms, ORM) are sufficient for building a complete, functional CRUD application without any third-party Django packages. The MVT pattern kept the code well-organized even as features were added incrementally. The combination of server-side rendering (Django templates) with targeted client-side interactivity (jQuery, AJAX, Canvas) is a practical middle ground for projects that do not justify a full single-page application framework.

For educational settings, a project like this is a good fit because it touches every layer of the stack. Students get to define database schemas, process forms, handle authentication, write server-side logic, build responsive UI, use JavaScript for dynamic behavior, and make AJAX calls — all in one codebase.

10. Conclusion

Diet & Calorie Tracker does what it set out to do: it gives users a fast, simple way to log meals, track daily calorie intake against a personal goal, and review their eating history. It is not trying to replace MyFitnessPal or Cronometer. It is a focused tool that solves one problem well and, in the process, demonstrates every core concept from the CSE 3243 Web Programming Lab syllabus.

Building this project taught us how the pieces of a web application fit together in practice, not just in isolated exercises. We learned to handle form conflicts on multi-form pages, to draw charts without a library, to structure Django views that serve both HTML and JSON, and to combine server-side filtering with client-side search for a responsive user experience. The codebase is modular enough that adding new features like macronutrient tracking or API integration would be straightforward, and the limitations we identified give clear direction for anyone who wants to take this project further.

References

[1] Django Software Foundation, “Django Documentation,” 2024. [Online]. Available: https://docs.djangoproject.com/. [Accessed: 2025].

[2] Bootstrap Team, “Bootstrap 5.3 Documentation,” 2024. [Online]. Available: https://getbootstrap.com/docs/5.3/. [Accessed: 2025].

[3] OpenJS Foundation, “jQuery API Documentation,” 2024. [Online]. Available: https://api.jquery.com/. [Accessed: 2025].

[4] MDN Web Docs, “Canvas API,” Mozilla, 2024. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API. [Accessed: 2025].

[5] L. E. Burke, J. Wang, and M. A. Sevick, “Self-Monitoring in Weight Loss: A Systematic Review of the Literature,” Journal of the American Dietetic Association, vol. 111, no. 1, pp. 92–102, Jan. 2011.

[6] Python Software Foundation, “Python 3.12 Documentation,” 2024. [Online]. Available: https://docs.python.org/3.12/. [Accessed: 2025].

[7] SQLite Consortium, “SQLite Documentation,” 2024. [Online]. Available: https://www.sqlite.org/docs.html. [Accessed: 2025].

[8] W3C, “HTML5 Specification,” 2024. [Online]. Available: https://html.spec.whatwg.org/. [Accessed: 2025].